HIGH-LEVEL DESIGN DOCUMENT

RAG-Based Customer Support Assistant

with LangGraph and Human-in-the-Loop

Prepared by: Kartik Patade

1. System Overview

1.1 Problem Definition

Most customer support systems either rely on rigid rule-based chatbots or require constant human intervention. Both approaches have their problems. Rule-based bots cannot handle queries that fall outside their predefined scripts. Fully human-operated support is expensive and slow at scale.

The goal of this project is to build a system that answers customer questions using information from an existing knowledge base (a PDF document), handles queries automatically when it can, and escalates to a human when it cannot. This is essentially what a RAG-based support assistant does.

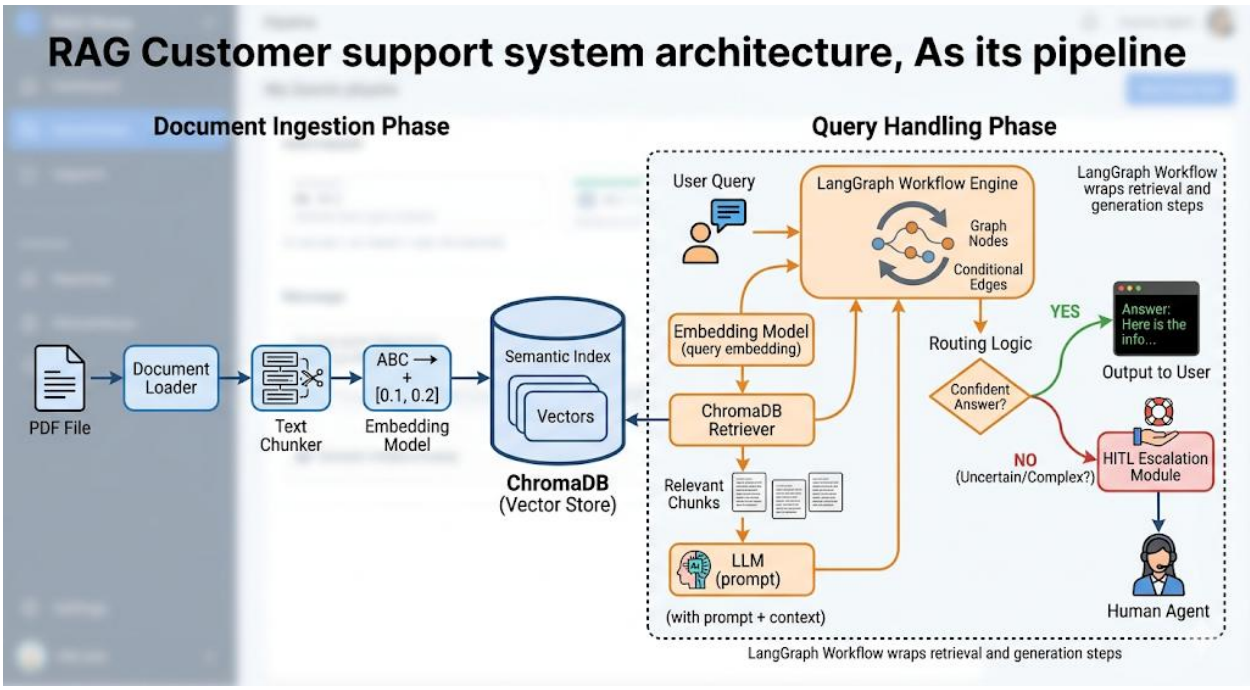
1.2 Scope

This system is designed to handle text-based customer queries. It reads a PDF file, processes it into searchable chunks, stores embeddings in a vector database, and retrieves relevant content at query time. A language model then uses that retrieved content to generate an answer. The workflow is managed using LangGraph. If the system is not confident in its answer, or if the query is too complex, it flags it for human review.

This is a single-document system. Multi-document support and persistent memory are listed as future improvements.

2. Architecture Diagram

The overall system follows a pipeline structure that can be broken into two main phases: document ingestion and query handling.



3. Component Descriptions

3.1 Document Loader

Uses PyPDFLoader from LangChain to load the PDF and extract the raw text from each page. It handles multi-page documents and outputs the text in a format that can be passed to the chunker.

3.2 Chunking Strategy

The text is split using RecursiveCharacterTextSplitter. Chunk size is set to 500 characters with an overlap of 50 characters. The overlap helps ensure that context is not lost at the boundary between two chunks. This size was chosen to keep chunks small enough to be meaningful but large enough to contain complete ideas.

3.3 Embedding Model

We use the sentence-transformers/all-MiniLM-L6-v2 model through HuggingFace. It is lightweight, runs locally without needing an API key, and produces good quality embeddings for retrieval tasks. Each chunk and each user query is converted into a 384-dimensional vector.

3.4 Vector Store

ChromaDB is used to store the embeddings. It runs in persistent mode, which means the embeddings are saved to disk and do not need to be regenerated every time the app starts. ChromaDB also handles the similarity search step when a query comes in.

3.5 Retriever

When a query arrives, it is embedded and compared against all stored vectors using cosine similarity. The top 3 most similar chunks are returned. These are passed as context to the LLM.

3.6 LLM

The system uses the Google Gemini API (gemini-1.5-flash) as the language model. It receives a prompt that includes the retrieved chunks and the user's question, and it generates a final answer. The prompt is designed to instruct the model to only answer based on the provided context and to say when it does not know.

3.7 Graph Workflow Engine (LangGraph)

LangGraph manages the flow of the query through the system. The graph has two main nodes: a retrieve_and_generate node and a route_response node. A state object is passed between nodes. Edges are conditional: if the LLM response contains low confidence signals, the graph routes to HITL escalation.

3.8 Routing Layer

The routing logic checks the LLM response for phrases like 'I don't know', 'cannot find', 'not sure', or if the retrieved chunks were empty. These are treated as escalation signals.

3.9 HITL Module

When escalation is triggered, the system logs the query and the attempted response, and prints a message indicating that a human agent needs to review it. In a production setting this would send a notification to a support queue. For this project it is kept simple and simulated via console output.

4. Data Flow

4.1 Ingestion Flow

- PDF file is loaded from disk
- Text is extracted page by page
- Text is split into chunks of ~500 characters
- Each chunk is embedded using MiniLM
- Embeddings and chunks are stored in ChromaDB

4.2 Query Lifecycle

- User inputs a question
- Query is embedded using the same MiniLM model
- Top 3 relevant chunks are retrieved from ChromaDB
- Chunks and query are formatted into a prompt
- LLM generates a response
- Response is passed to the router
- Router decides: send to user OR escalate to human

5. Technology Choices

5.1 Why ChromaDB

ChromaDB is easy to set up locally and works well for small to medium sized knowledge bases. It does not require any cloud account or paid plan. Persistent storage means embeddings are reused across sessions without re-running the ingestion pipeline. For this project scale it is more than sufficient.

5.2 Why LangGraph

LangGraph provides a clean way to structure the workflow as a graph with states and conditional transitions. It is built by the same team as LangChain, so it integrates naturally with all the components already being used. It also makes the routing logic explicit and easy to modify.

5.3 LLM Choice: Gemini 1.5 Flash

Gemini 1.5 Flash is available through a free-tier API and handles large context windows well. It is fast for response generation and reliable. It was chosen mainly for availability and cost, since this is an internship project.

5.4 Other Tools

LangChain is used for loading and splitting documents and for building the retrieval chain. HuggingFace Transformers provides the embedding model. Python is used throughout.

6. Scalability Considerations

6.1 Handling Large Documents

The current chunk size and overlap work fine for a typical support document. For much larger PDFs, it would help to use a semantic chunker that splits based on paragraph or sentence boundaries rather than fixed character count. ChromaDB can scale to millions of vectors without much configuration change.

6.2 Increasing Query Load

For higher query volumes, the embedding generation step would need to be moved to a background queue rather than done synchronously. ChromaDB can also be run as a separate service rather than embedded in the application process.

6.3 Latency Concerns

Currently the main sources of latency are the embedding step and the LLM call. Using a quantized local model instead of a remote API could reduce round-trip time. Caching common query embeddings is another option for repeated queries.